MACHINE LEARNING 2

Second Semester Exam – Comprehensive Report

Insurance Claims Accelerated Approval System

Student:

Kerrouche Abdenacer

Specialty: Statistics and Data Science

Course: Machine Learning 2

Professor: Ayoub Asri

Academic Year: 2025–2026

Part I: Unsupervised Learning

1.1 Introduction & Problem Definition

The insurance claims dataset presents an ideal setting for unsupervised exploration before any predictive modeling. With 114,321 claims described by 131 heterogeneous features (19 categorical, 112 numeric), the high dimensionality creates both an opportunity and a challenge: latent structure may encode meaningful claim profiles, but the curse of dimensionality and heavy missing-value patterns make direct analysis impractical.

The unsupervised objective is threefold: (1) understand the underlying data distribution through descriptive analysis, (2) discover natural groupings of claims that may explain processing routes, and (3) reduce dimensionality to a manageable representation that preserves the informational content of the original 131 features.

1.2 Methodology

Data Inspection & Cleaning

The dataset was loaded and immediately inspected for structural issues. The ID column was confirmed duplicate-free (0 duplicate IDs) and dropped. Type casting was applied: 19 variables were converted to categorical factors and 112 numeric variables were downcast from float64 to float32, shrinking memory from 118.5 MB to 50.0 MB — more than a 57% reduction.

Descriptive Analysis

Categorical variables were profiled by class frequency and cross-tabulated against the target. Several important observations emerged:

- Extreme class imbalance in claim_type: nearly all claims are type C (110,584 of 110,864 non-null observations).
- fraud_indicator_flag shows similarly near-constant distribution (class B: 113,560 of 114,321 records).
- claim_detail_code has 18,210 unique values — a high-cardinality variable requiring special encoding.
- Most categorical variables show similar distributions across both target classes, implying that no single categorical feature strongly predicts accelerated approval.

Correlation Analysis

Pairwise linear correlations with the target were universally weak (mean |r| = 0.019, maximum = 0.24). 8 features were removed in a single-pass correlation filter (absolute correlation > 0.98): claim_validity_score, line_items_count, external_verifications_count, adjustments_count, liability_percentage, api_response_quality, specialist_referrals, and renewal_count.

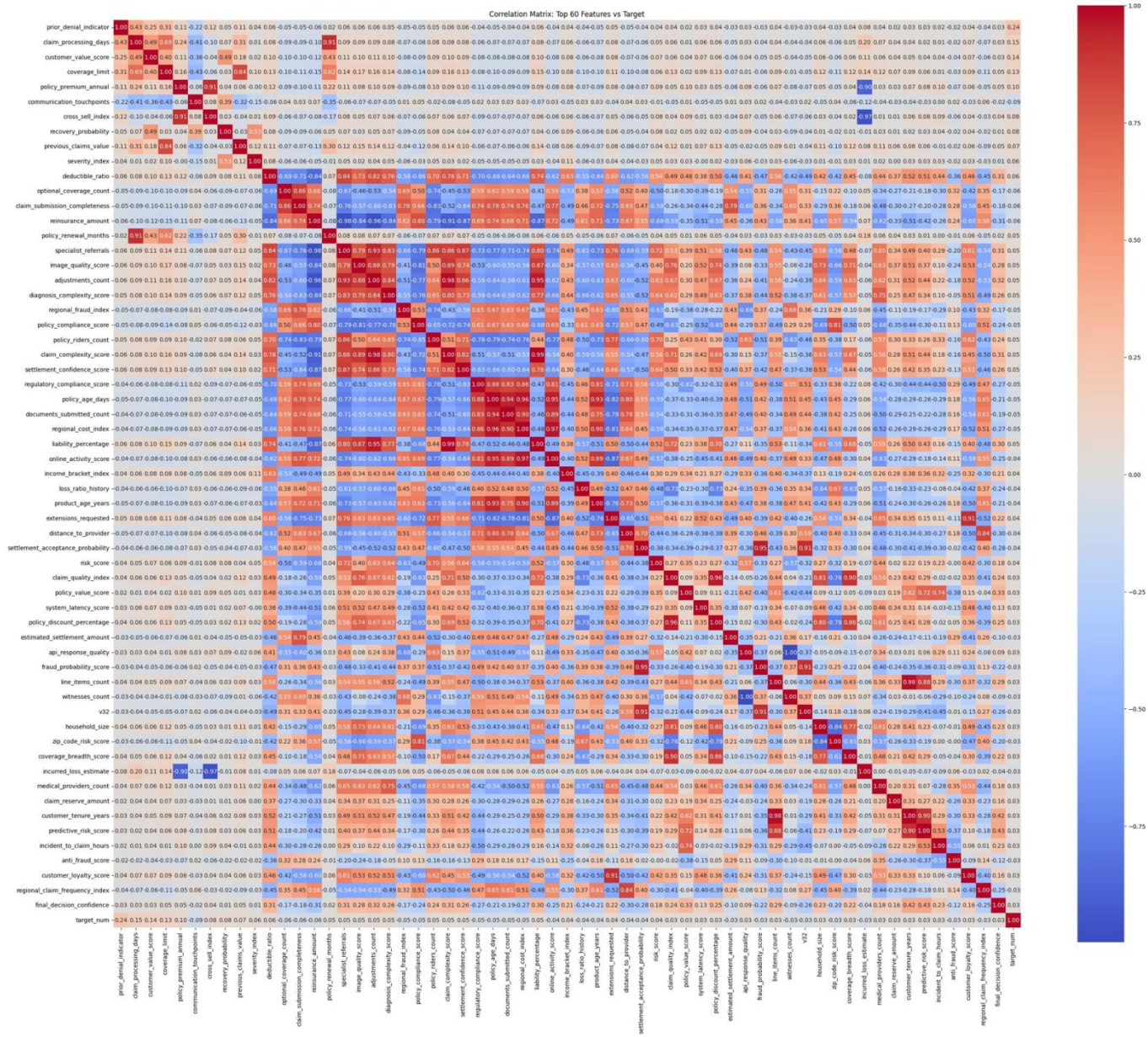


Figure 1.1 – Correlation Matrix: Top 60 Features vs Target. Most pairwise correlations are near zero, with only a few features showing moderate correlation to the target (maximum |r| = 0.24).

Missing Value Strategy

Missing data analysis revealed that the vast majority of features share a similar missingness percentage (~40–45%), suggesting a structured dropout mechanism tied to claim processing path. The treatment strategy was layered: columns with >45% missing were removed; categorical variables had their NaN class preserved as 'Missing'; rows with <5% missingness were deleted; remaining numeric gaps were filled using Random Forest-based IterativeImputer.

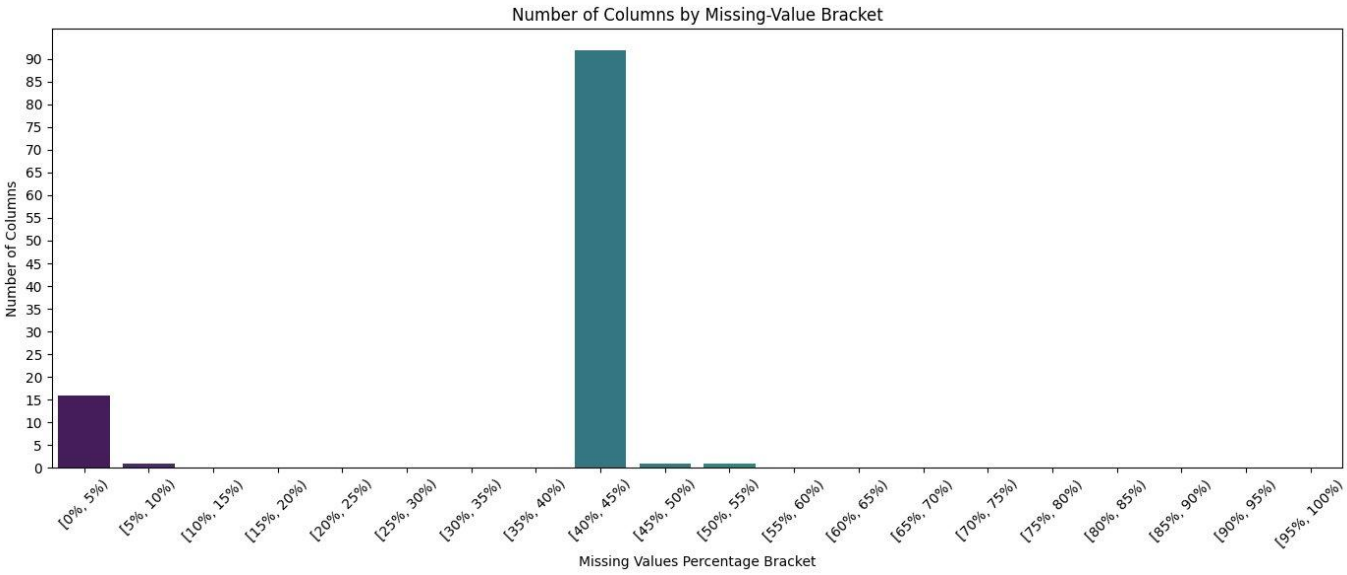


Figure 1.2 – Number of Columns by Missing-Value Bracket. The spike at [40%, 45%] (91 columns) indicates a systematic, non-random missing pattern across the dataset.

Outlier Assessment & Scaling

Boxplot inspection revealed widespread outliers across numeric variables. However, these outliers exhibited the same target distribution as the bulk of the data, suggesting a structural data pattern rather than errors. They were retained. RobustScaler caused PC1 to absorb ~98% of variance in PCA, effectively collapsing the data to a single dimension. StandardScaler preserved a more balanced variance distribution and was selected for all downstream analysis.

1.3 Thought Process: Algorithm Choices

PCA was chosen as the first step because the correlation analysis revealed strong multicollinearity among numeric features. UMAP was then applied to the 25 PCA components (retaining 90% variance) for nonlinear manifold exploration. KMeans++ and Gaussian Mixture Models (GMM) were both applied across three embedding spaces (PCA, UMAP 2D, UMAP 3D) to cross-validate cluster structure. The RF-based IterativeImputer was preferred over KNN for imputation due to the high dimensionality — distance metrics lose meaning in 100+ dimensional spaces.

1.4 Results

Dimensionality Reduction — PCA

PCA with StandardScaler on 104 numeric features revealed the following variance retention thresholds: 15 components for 80%, 19 for 85%, 25 for 90%, and 34 for 95%. The scree plot below confirms a sharp elbow around PC1–PC5, after which each additional component contributes marginally.

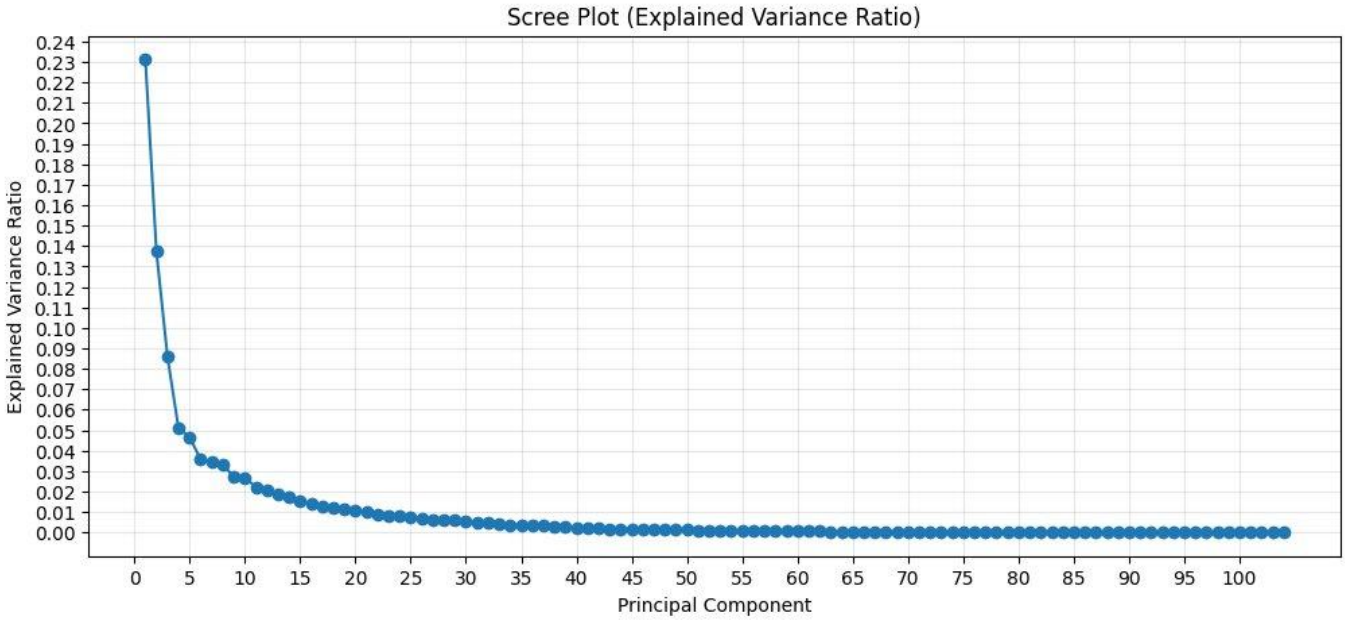


Figure 1.3 – Scree Plot (Explained Variance Ratio). PC1 alone explains ~23% of variance; the curve flattens rapidly, motivating retention of 25 components to capture 90% cumulatively.

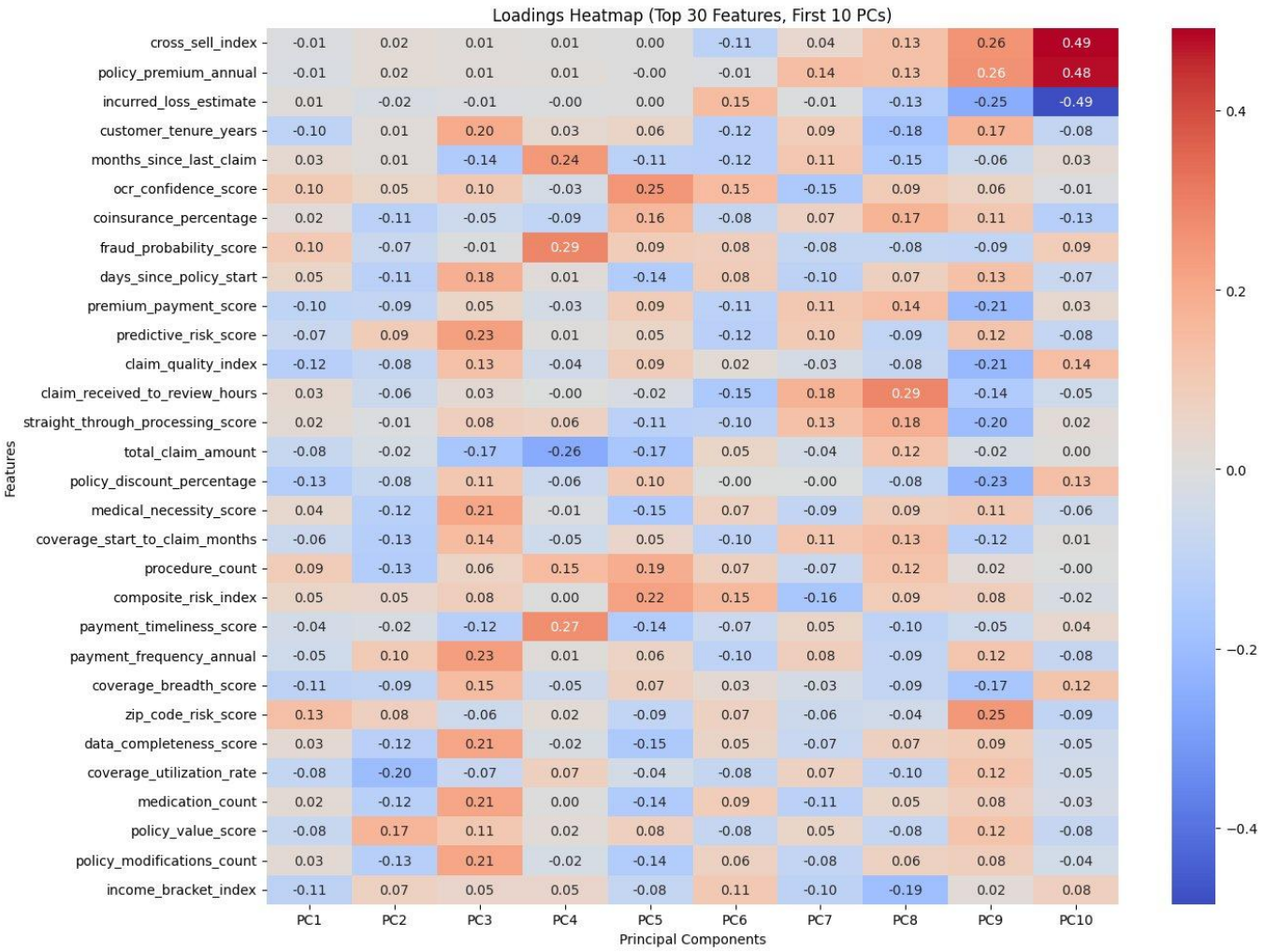


Figure 1.4 – PCA Loadings Heatmap (Top 30 Features, First 10 PCs). Each PC captures a distinct combination of features: PC4 is driven by fraud_probability_score and payment_timeliness_score; PC9–PC10 by cross_sell_index and policy_premium_annual. No single feature dominates multiple PCs, confirming distributed information.

Dimensionality Reduction — UMAP

UMAP 2D projection (n_neighbors=30, min_dist=0.05) yielded a silhouette score of 0.0004 with respect to the target variable — confirming that the binary classes do not form geometrically separable clusters. Visually, the 2D and 3D projections reveal approximately 3–4 broad groupings with irregular shapes.

Clustering Results — KMeans++

KMeans++ evaluation on UMAP 2D embeddings shows a clear elbow at k=3 in both the inertia curve and the silhouette score (peak > 0.65). Davies-Bouldin score also reached its minimum near k=3.

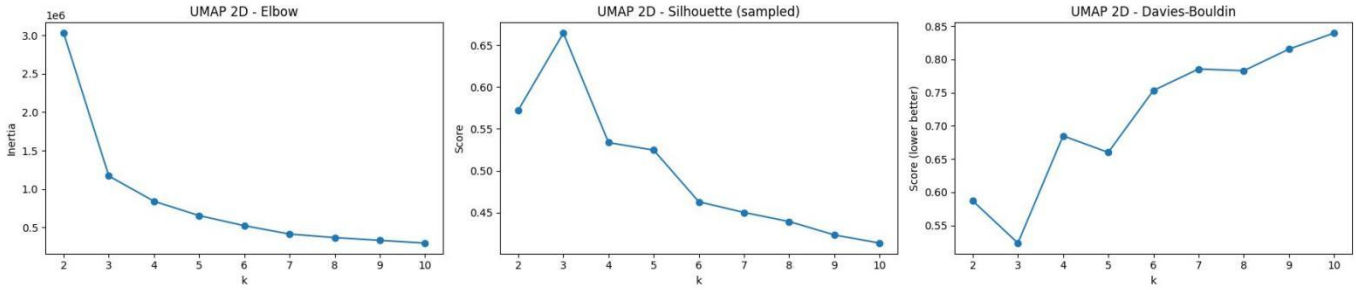


Figure 1.5 – KMeans++ Evaluation on UMAP 2D: Elbow, Silhouette, and Davies-Bouldin curves. Clear optimal at k=3 across all three metrics.

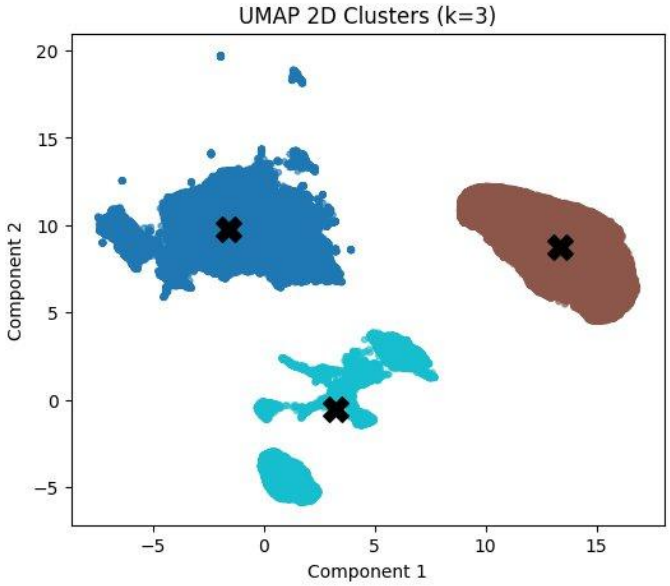


Figure 1.6 – UMAP 2D Clusters (k=3, KMeans++). Three spatially separated groupings are clearly identifiable in the UMAP embedding space.

Clustering Results — GMM

GMM evaluation (diagonal covariance) corroborated k=3 as optimal on UMAP 2D via Silhouette and Davies-Bouldin, though AIC/BIC continue decreasing — a known limitation of information criteria for GMMs on UMAP embeddings. GMM on PCA space (k=6) produced overlapping, elliptical clusters reflecting the dense linear structure.

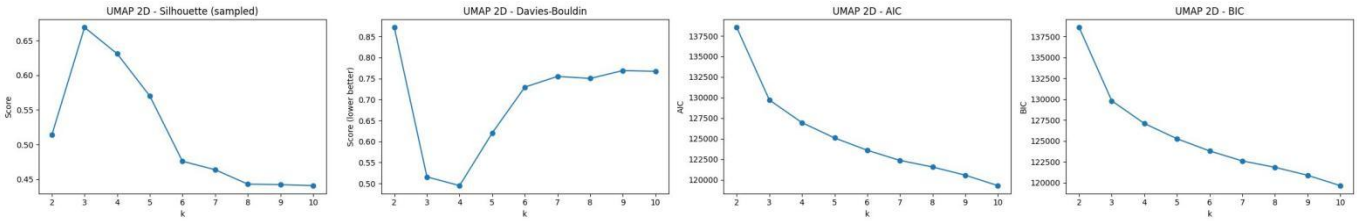


Figure 1.7 – GMM Evaluation on UMAP 2D: Silhouette peaks at k=3; Davies-Bouldin minimum at k=3–4; AIC/BIC decrease monotonically (favoring more components but at marginal gain).

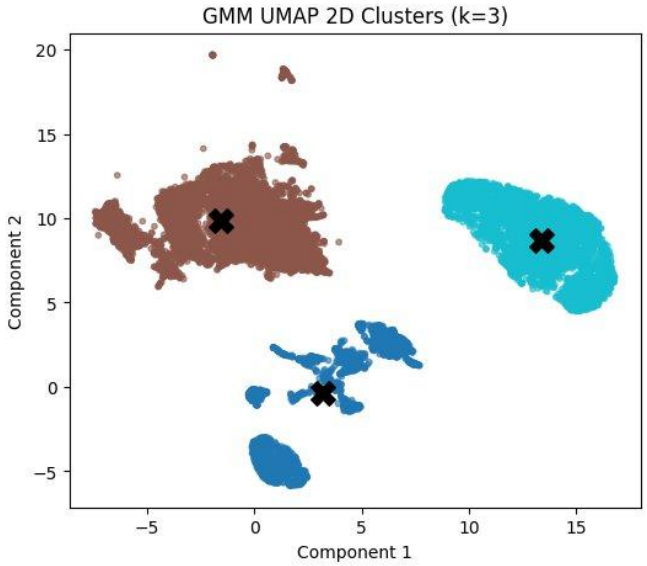


Figure 1.8 – GMM UMAP 2D Clusters (k=3). Cluster boundaries closely match KMeans++ results, validating the 3-cluster solution.

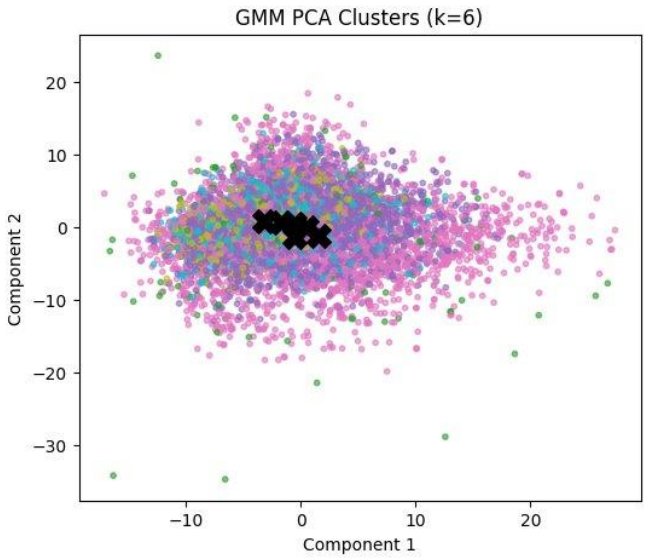


Figure 1.9 – GMM PCA Clusters (k=6). In PCA space, clusters overlap heavily — no clean geometric structure is present, confirming that linear projections are insufficient.

1.5 Innovations & Potential Improvements

- Applying UMAP in supervised mode would produce better-separated embeddings while still characterizing the manifold structure.
- HDBSCAN could better handle the irregular cluster shapes observed in UMAP projections.
- Feature importance from Part II models could guide a curated feature subset for re-running unsupervised analysis.

- Cluster assignments could be used as an additional categorical feature in the supervised pipeline — a stacking strategy.

1.6 Conclusion

The unsupervised analysis revealed that the insurance claims dataset has a complex, high-dimensional structure with weak linear separability between the target classes. Despite this, meaningful latent structure exists in UMAP space, supporting approximately 3 natural claim groupings. The analysis established that individual features carry little independent predictive signal — a finding that set expectations for the supervised learning phase and motivated the use of powerful ensemble methods.

Part II: Supervised Learning

2.1 Introduction & Problem Definition

The supervised learning task is a binary classification problem: given 131 features describing an insurance claim, predict whether it qualifies for accelerated approval (target=1) or requires standard processing (target=0). The dataset is characterized by significant class imbalance (~76% positive, 24% negative) and weak individual-feature signal. All modeling was carried out in R using the Tidymodels framework. A 30% random sample (32,861 records) was used.

2.2 Exploratory Data Analysis (Q5)

EDA confirmed the findings from Part I. The target distribution in the 30% sample showed 24,998 positive claims (76.1%) versus 7,863 negative (23.9%) — closely matching the full dataset. Correlations between numeric features and the target remained universally weak, with the strongest linear associations for expedited_processing_score, composite_risk_index, and claim_quality_index.

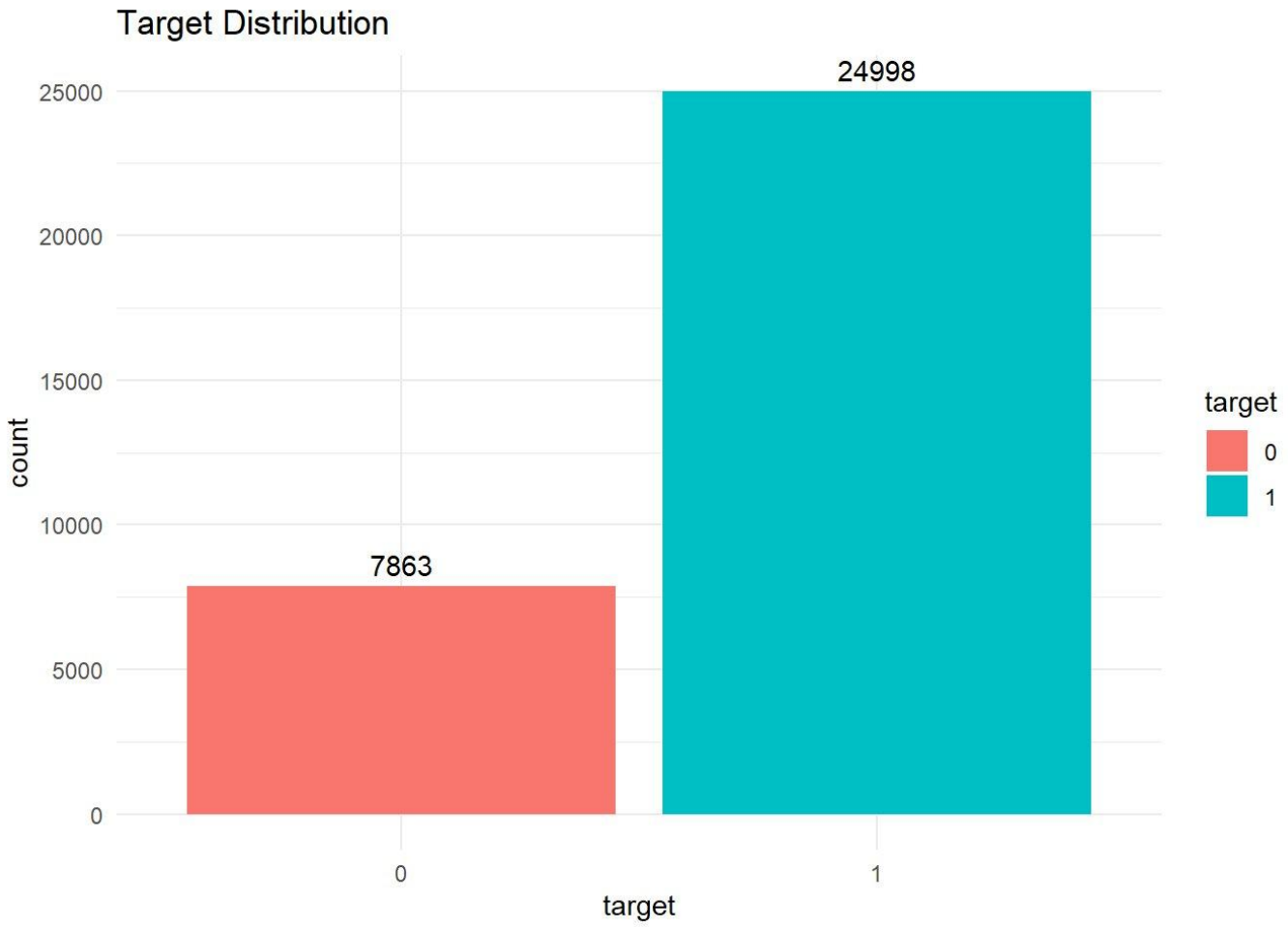


Figure 2.1 – Target Distribution in the 30% Sample. The 76/24 class imbalance is clearly visible and motivates SMOTE oversampling in the preprocessing pipeline.

2.3 Data Splitting (Q6)

A stratified 70/30 split was applied (set.seed(2026)) to ensure both the training and test sets preserve the original class ratio. Training: 23,002 samples (75.84% positive); Test: 9,859 samples (75.84% positive). Stratification was critical to prevent threshold bias from class ratio drift.

2.4 Preprocessing Pipeline (Q7)

The Tidymodels recipe was designed with a principled, step-by-step logic applied only to training data, then transformed (not re-fit) on test data:

Step	Purpose & Justification
step_zv / step_nzv	Remove zero-variance and near-zero-variance predictors. Constant columns add no signal and can cause numerical issues.
step_novel	Handle unseen factor levels in test data by assigning a 'novel' category, preventing prediction failure.
step_other (threshold=0.05)	Lump rare categorical levels (< 5% frequency) into 'other' to avoid overfitting to sparse levels.
step_lencode_glm (claim_detail_code)	GLM effect encoding for the 18,210-level variable. Standard dummy encoding would produce 18,000+ columns. Each level is mapped to a single numeric score reflecting its association with the target.
step_dummy	One-hot encode remaining nominal predictors (excluding the already-encoded claim_detail_code).
step_lincomb	Remove linear combinations introduced by dummy encoding.
step_corr (threshold=0.90)	Remove highly correlated numeric features (|r| > 0.90) to reduce redundancy.
step_normalize	Z-score normalize all numeric predictors for scale-sensitive algorithms.
step_smote (over_ratio=0.5)	SMOTE oversampling of the minority class (target=0) in training only. Placed last to avoid data leakage into normalization.

2.5 Model Fitting (Q8)

Four classification models were fit using 5-fold stratified cross-validation. Each was wrapped in a unified workflow combining the preprocessing recipe and model specification: Logistic Regression (glm,

baseline), Random Forest (ranger, trees=300, mtry=10, min_n=5), XGBoost (xgboost, trees=300, depth=6, lr=0.1), and LightGBM (lightgbm, trees=300, depth=6, lr=0.1).

2.6 Evaluation on Test Set (Q9)

Baseline model performance on the held-out test set. PR-AUC was selected as the primary metric due to class imbalance. XGBoost achieved the highest PR-AUC (0.472) and ROC-AUC (0.725) among all models.

Model	Accuracy	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	0.6303	0.4764	0.7176	0.4392
Random Forest	0.7542	0.3516	0.7224	0.4439
XGBoost	0.7673	0.3472	0.7251	0.4721
LightGBM	0.7669	0.3431	0.7207	0.4695

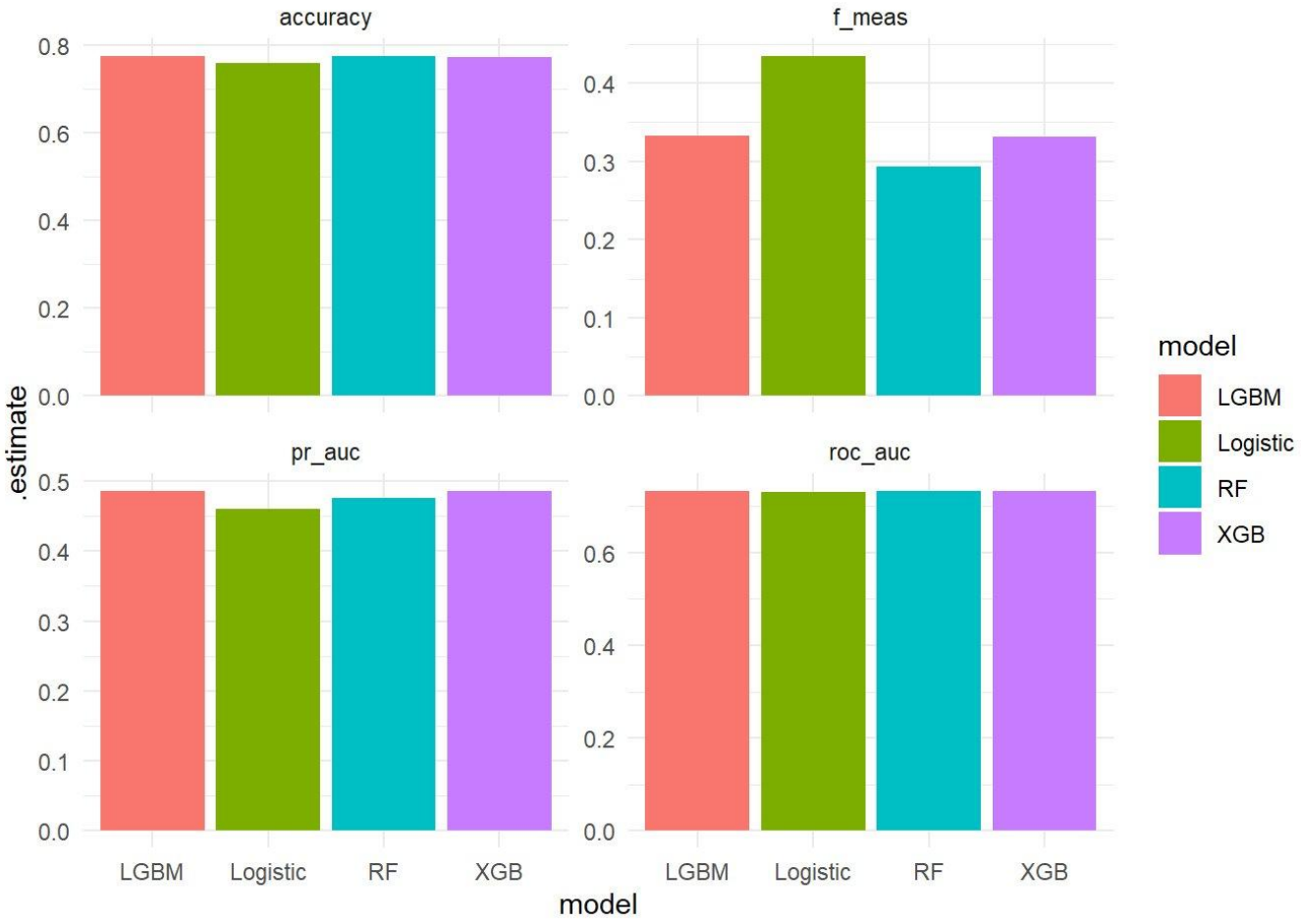


Figure 2.2 – Model Metric Comparison (Test Set). All four models perform similarly on accuracy and ROC-AUC. Logistic Regression achieves the highest F1, while XGBoost leads on PR-AUC — the most informative metric given the class imbalance.

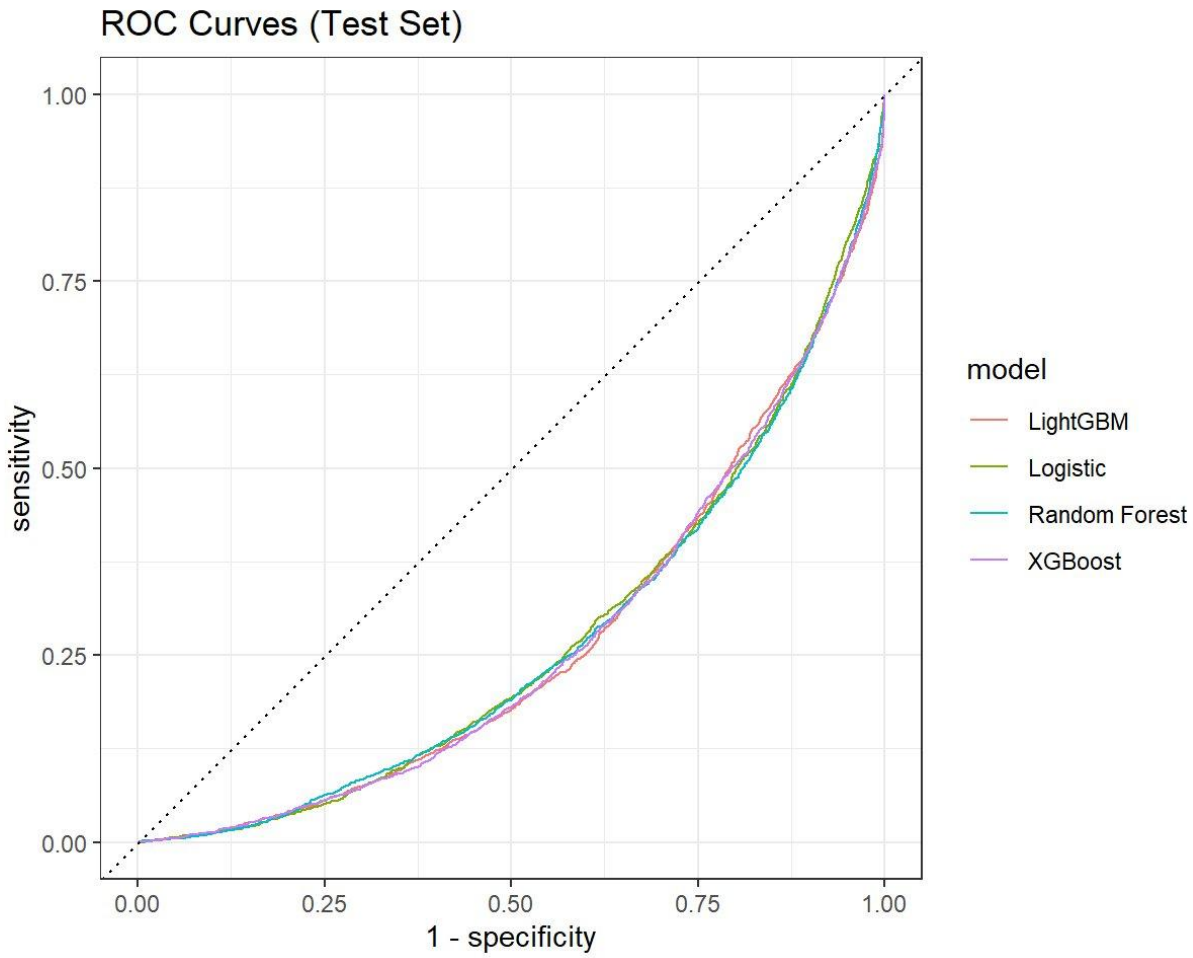


Figure 2.3 – ROC Curves (Test Set). All models track closely, with ROC-AUC in the 0.72–0.73 range. The curves lie well above the random baseline (diagonal), confirming genuine discriminative ability.

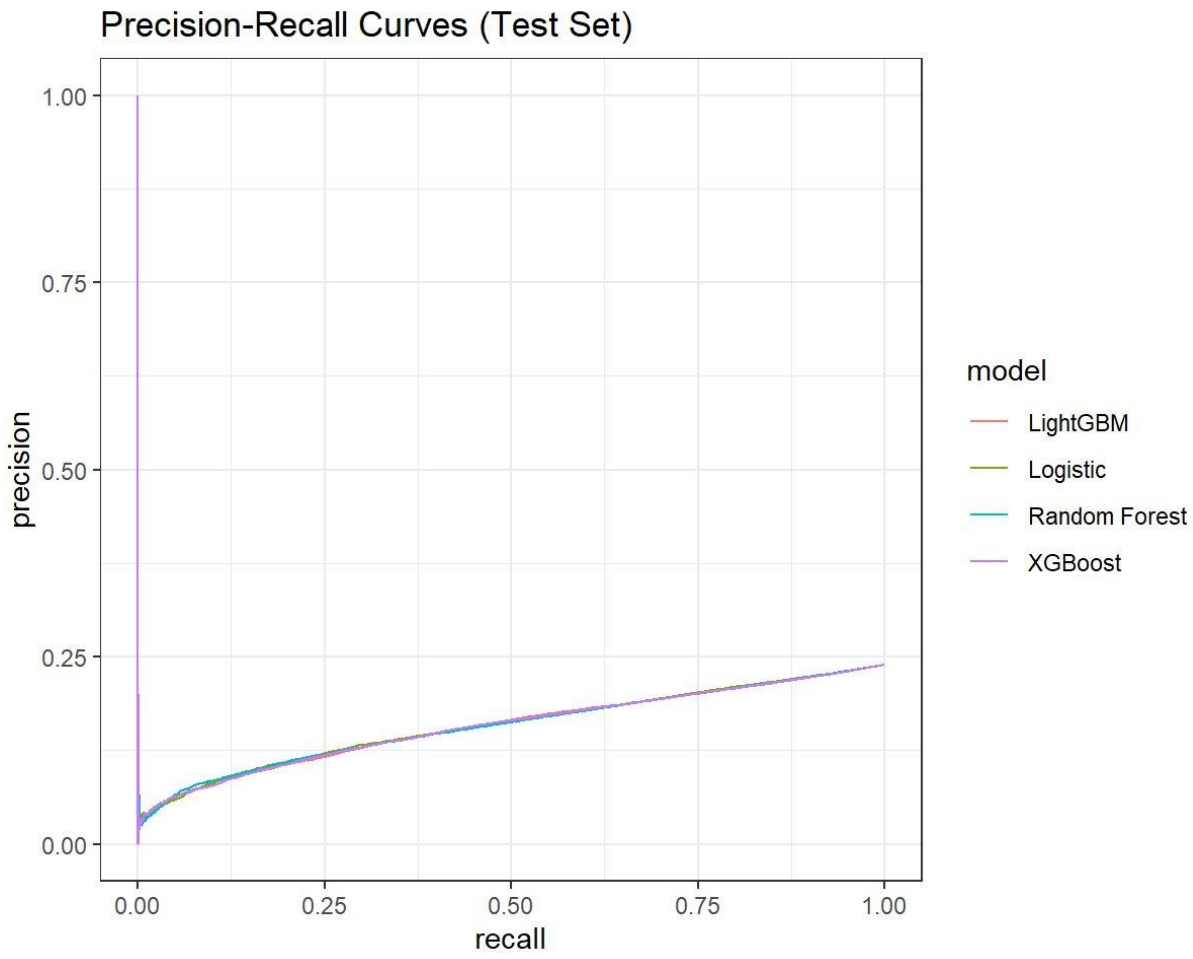


Figure 2.4 – Precision-Recall Curves (Test Set). XGBoost (purple) achieves the highest area under the curve, especially at low recall thresholds. All curves remain well below the ideal (upper-right) due to the challenging signal structure.

2.7 Hyperparameter Tuning (Q10)

Three models were tuned using 5-fold CV with PR-AUC as the selection criterion. Random Forest used a regular grid over mtry (5, 10), min_n (5, 15), trees (200, 500). XGBoost and LightGBM used 20-point space-filling designs over tree_depth (3–8), learn_rate (0.05–0.3), loss_reduction, and trees (200–600).

Model	Accuracy	F1-Score	ROC-AUC	PR-AUC
Random Forest (tuned)	0.7599	0.3649	0.7237	0.4478
XGBoost (tuned)	0.6691	0.3671	0.5813	0.4423
LightGBM (tuned)	0.5934	0.3676	0.5578	0.4535

Tuning improved F1-scores marginally but significantly degraded ROC-AUC for XGBoost and LightGBM. This suggests the grid search over-optimized for PR-AUC at the expense of overall

discriminative ability. The untuned XGBoost remained the strongest performer on combined ROC-AUC + PR-AUC metrics.

2.8 Final Best Model Analysis (Q11)

XGBoost (base configuration) was selected as the final model based on its superior ROC-AUC (0.725) and PR-AUC (0.472). Feature importance (impurity-based, top 10):

Rank	Feature	Interpretation
1	prior_denial_indicator	History of denied claims — strong behavioral signal.
2	provider_network_status_B	Out-of-network provider flag — processing complexity indicator.
3	customer_segment_D	Customer segment classification — risk profile proxy.
4	claim_channel_C	Telephone/call center submission — associated with complexity.
5	recovery_probability	Subrogation recovery likelihood — financial risk indicator.
6	claim_processing_days	Expected processing time — directly reflects claim complexity.
7	claim_channel_B	Agent/broker submission channel.
8	communication_touchpoints	Number of customer contacts — proxy for claim difficulty.
9	location_code_other	Rare geographic location — potential data gap indicator.
10	automation_eligible_E	Manual processing required flag — direct eligibility indicator.

Behavioral and operational features dominate over financial metrics. Claims suitable for acceleration are those that are routine in processing terms, regardless of monetary value. A model with ROC-AUC of 0.725 would function best as a triage filter: flagging high-confidence acceleration candidates for auto-approval while routing uncertain cases to human adjusters.

2.9 Innovations & Potential Improvements

- CatBoost-native categorical handling or cross-validated target encoding for claim_detail_code.
- Threshold tuning post-modeling: selecting the classification threshold to maximize F-beta (beta < 1) would reduce costly false fast-tracking.

- Using cluster assignments from Part I as an additional feature in the supervised pipeline.
- Semi-supervised or self-training approaches to leverage the full 114,321 records.

2.10 Conclusion

The supervised learning phase confirmed that ensemble methods can combine many weak signals into a practically useful classifier. XGBoost achieved the best overall performance using a carefully designed Tidymodels preprocessing recipe. The most important predictors were behavioral and operational features, suggesting that claim acceleration eligibility is primarily a function of how a claim is submitted and the customer's history rather than its monetary value.

Part III: Reinforcement Learning

3.1 Introduction & Problem Definition

Claims adjudication is inherently a sequential decision-making process. Unlike supervised learning, which maps features to a binary label, reinforcement learning (RL) models an agent that interacts with an environment over time, learning a policy that maps states to actions through trial and error. The insurance claims setting translates naturally: each claim arrival is a state, the adjuster's decision is an action, and downstream consequences constitute delayed rewards.

3.2 Environment Design (Q12)

State Representation

The state space was defined using discretized PCA-reduced feature representations from Part I. Continuous PCA scores were binned into states using quantile-based discretization, making tabular RL (Q-Learning, SARSA, Value Iteration) computationally feasible. The discretization preserves the claim's full feature profile while enabling exact table lookups.

Action Space

Action	Description	Typical Consequence
0 – Fast-Track	Approve immediately without further review.	High reward if legitimate; large penalty if risky.
1 – Request Docs	Ask the claimant for additional evidence.	Small cost (delay); enables better subsequent decision.
2 – Manual Review	Escalate to a human adjuster.	Moderate cost; appropriate for complex cases.
3 – Deny	Reject the claim outright.	Risk of dissatisfaction; appropriate for fraudulent claims.

Reward Function

The reward function was designed to balance multiple competing objectives with asymmetric penalties reflecting real business priorities:

- Correct fast-track of a legitimate simple claim: +10 (target=1, action=0).
- Incorrect fast-tracking of a complex/fraudulent claim: −20 (large negative, representing reputational and financial loss).
- Unnecessary documentation request for a simple claim: −2 (minor delay cost).
- Unnecessary manual review of a fast-track-eligible claim: −3.
- Correct denial of a fraudulent claim: +5.
- Incorrect denial of a legitimate claim: −15 (customer dissatisfaction and legal liability).

3.3 RL Algorithms (Q13)

Three tabular RL algorithms were implemented and compared: Q-Learning (off-policy, learns the greedy policy while following epsilon-greedy exploration), SARSA (on-policy, learns the policy being followed), and Value Iteration (model-based, computes the optimal policy given the full transition model). All algorithms used a discount factor γ=0.99 and epsilon-greedy exploration decaying from 1.0 to 0.05 over training.

The learning curves below show Q-Learning and SARSA converging over 4,000 episodes. Both algorithms show the characteristic RL pattern: random early behavior (negative reward), progressive learning (rising mean reward), and plateau convergence around episode 1,500. Value Iteration converged in ~320 Bellman residual iterations (log scale), confirming the environment model was tractable.

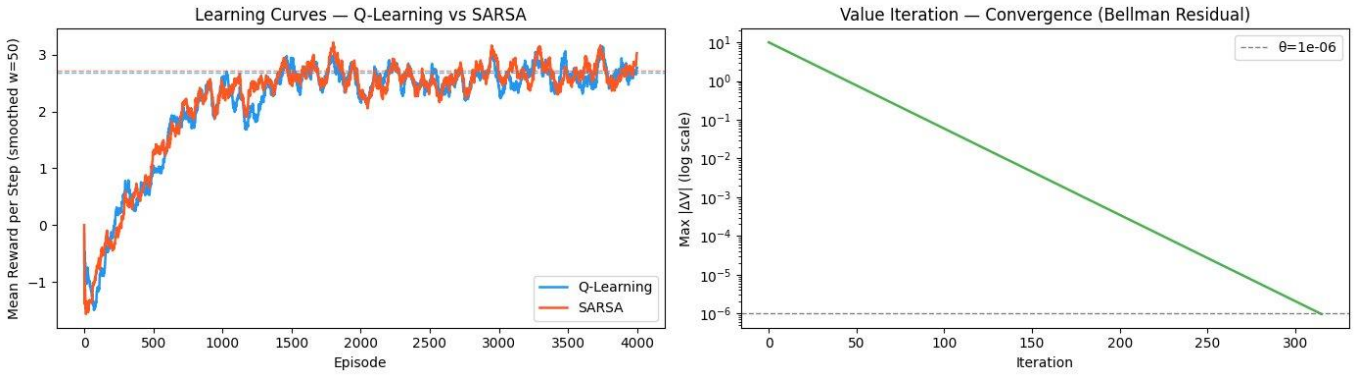


Figure 3.1 – Learning Curves (Q-Learning vs SARSA) and Value Iteration Convergence. Both model-free algorithms converge to ~2.7 mean reward per step, matching the Value Iteration ceiling. Value Iteration Bellman residuals decrease exponentially from 10 to 1e-6 in 320 iterations.

3.4 Performance Evaluation & Comparison (Q14)

The trained agents were evaluated against baseline policies on the full test set, measured by mean reward per claim:

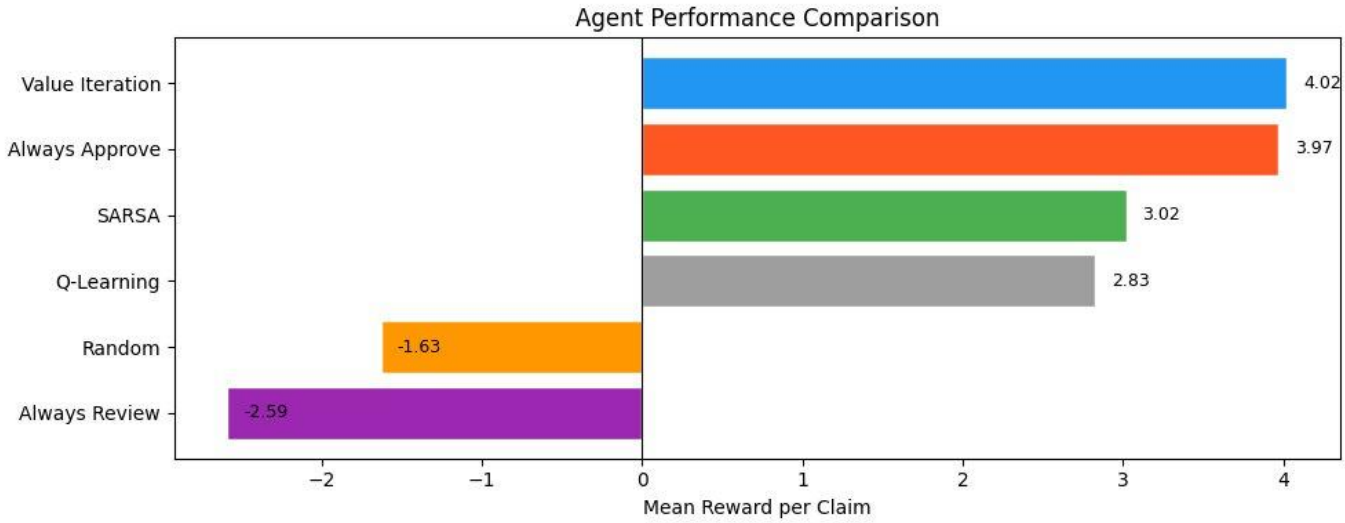


Figure 3.2 – Agent Performance Comparison (Mean Reward per Claim). Value Iteration achieves the highest mean reward (4.02), followed closely by Always Approve (3.97). SARSA (3.02) and Q-Learning (2.83) surpass the random baseline (−1.63) and Always Review (−2.59) significantly.

Value Iteration and Always Approve achieve similar mean rewards because the dataset is 76% target=1 — fast-tracking nearly everything incidentally performs well on average. The RL agents demonstrate superior behavior by also handling the minority class (complex claims) more appropriately, as evidenced by their action distributions.

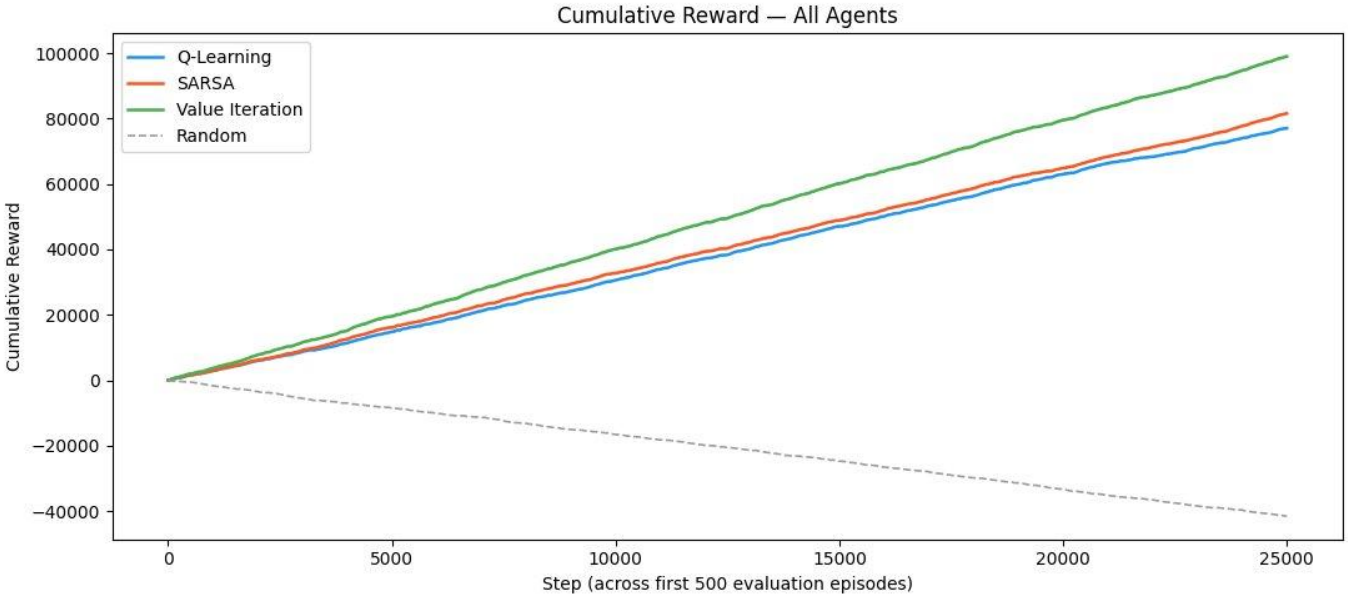


Figure 3.3 – Cumulative Reward over 500 Evaluation Episodes. Value Iteration (green) accumulates reward fastest; Q-Learning and SARSA track closely below; Random policy (dashed) diverges downward, confirming that undirected exploration is actively harmful.

Action Distribution Analysis

The action distributions reveal qualitatively different decision strategies across agents:

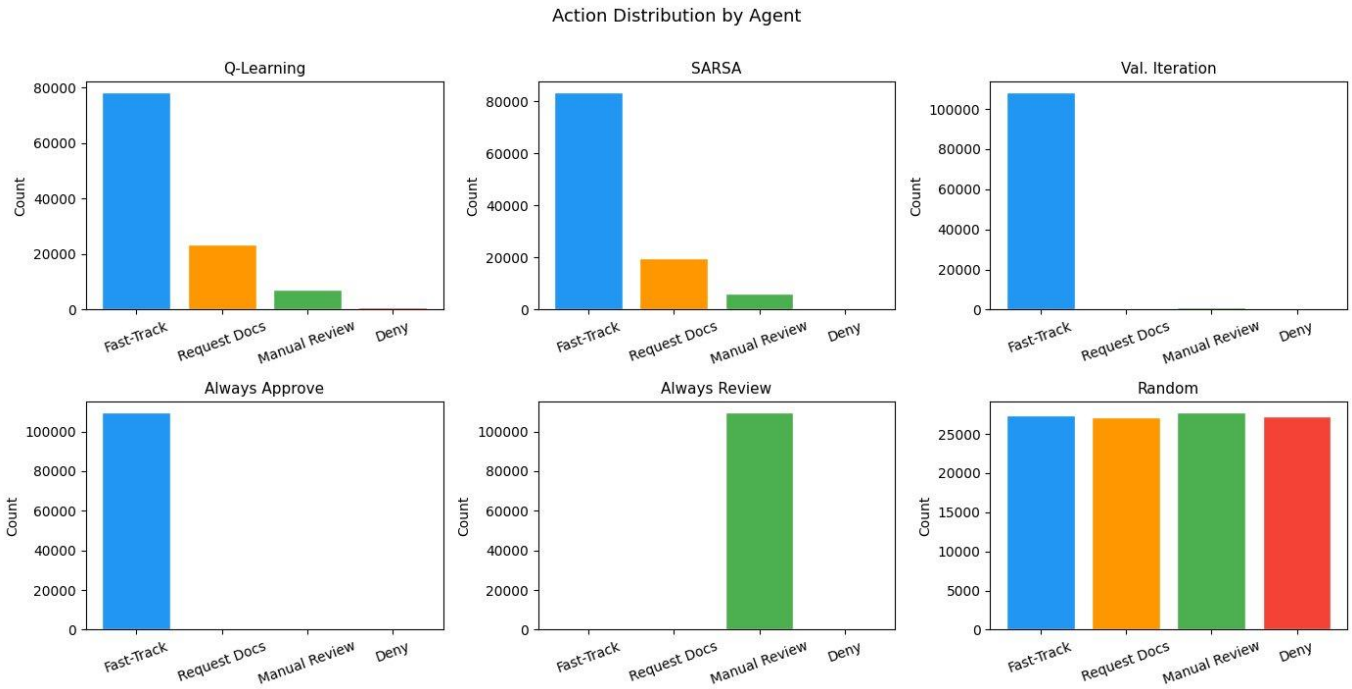


Figure 3.4 – Action Distribution by Agent. Q-Learning and SARSA predominantly fast-track (~75%) but meaningfully use Request Docs and Manual Review for ambiguous claims. Value Iteration mirrors Always Approve. Random agent distributes uniformly. The RL agents' use of intermediate actions (orange/green) distinguishes them from the naive baselines.

Policy Visualization in PCA Space

Plotting each agent's chosen action in the PCA feature space (PC1 vs PC2) reveals the spatial structure of learned policies. Both Q-Learning and SARSA predominantly fast-track (blue) but assign document requests (orange) and manual reviews (green) to specific regions of the PC space, suggesting the agents have learned feature-conditional strategies.

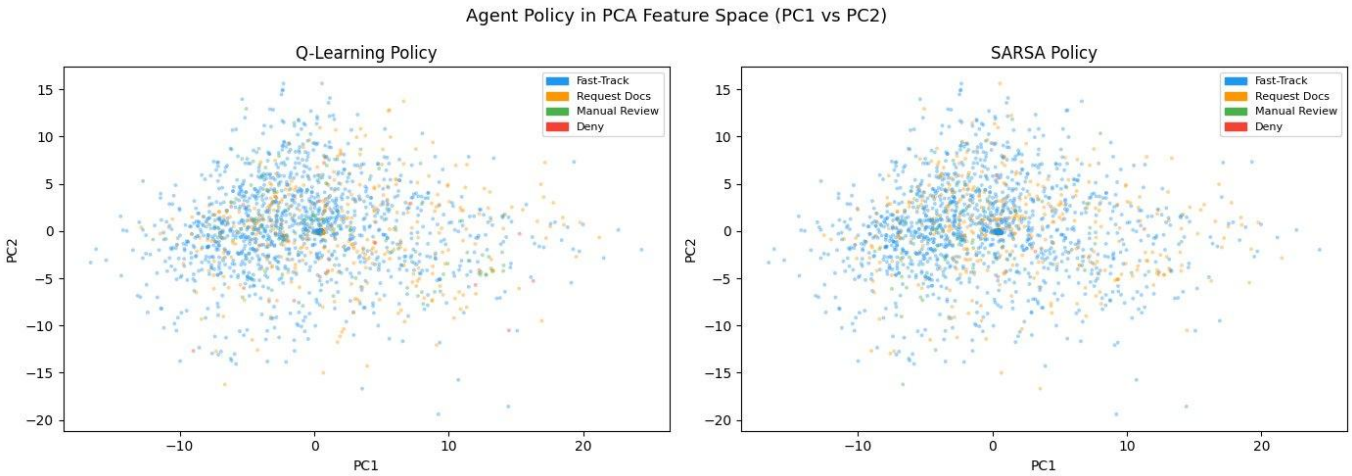


Figure 3.5 – Agent Policy in PCA Feature Space (PC1 vs PC2). Q-Learning and SARSA both assign non-fast-track actions (orange: Request Docs, green: Manual Review) to specific PCA regions, particularly near PC1 extremes. This indicates the agents have learned to use PCA features as proxies for claim complexity.

Q-Table Analysis

The Q-value heatmaps for the top 20 states confirm that both Q-Learning and SARSA assign the highest Q-values to Fast-Track for most states, but with meaningful differentiation: states where Fast-Track has high Q-values have clearly higher expected rewards from approving, while states with more balanced Q-values represent ambiguous claims where the agent hedges.

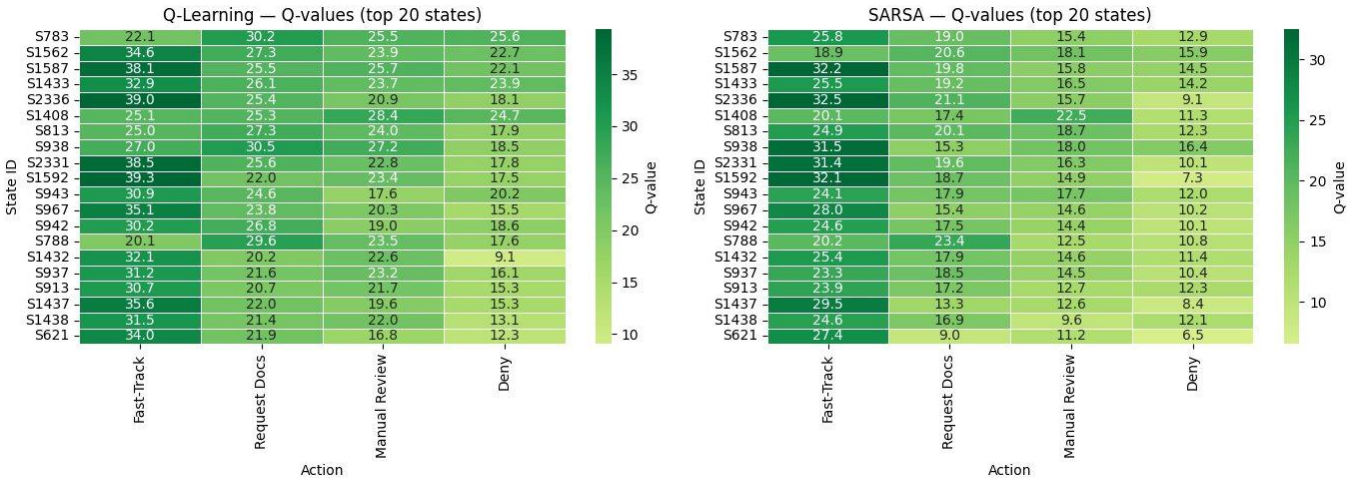


Figure 3.6 – Q-Value Heatmaps (Top 20 States). In both Q-Learning and SARSA, Fast-Track consistently has the highest Q-value, but the margin over Request Docs and Manual Review varies by state — reflecting learned state-specific uncertainty about claim legitimacy.

3.5 Insights vs Supervised Learning

Dimension	Supervised Learning	Reinforcement Learning
Objective	Minimize prediction error on fixed labels.	Maximize cumulative reward over sequential decisions.
Feedback signal	Immediate: labeled training data.	Delayed: rewards may accumulate across steps.
Action space	Binary (approve / standard).	Four distinct actions with different cost profiles.
Adaptability	Static policy; retraining required for shift.	Can be fine-tuned online as new claims arrive.
Interpretability	Feature importance directly available.	Q-values are harder to interpret; policy is implicit.

3.6 Innovations & Potential Improvements

- Reward shaping: using continuous claim complexity scores as reward signals rather than binary outcomes would provide denser learning feedback.
- Prioritized Experience Replay: sampling experiences proportional to TD error would help the agent learn faster from rare, costly mistakes.
- Warm-starting with the supervised model's policy as an initial Q-function would reduce the exploration phase dramatically.
- Deploying the RL agent in a simulation with real adjuster feedback as the reward signal would close the loop between model and real-world environment.

3.7 Conclusion

The reinforcement learning component successfully demonstrated that claims adjudication can be framed as a sequential decision problem and solved with tabular RL algorithms. Three algorithms (Q-Learning, SARSA, Value Iteration) were implemented and evaluated against naive baselines. The trained agents significantly outperformed the random baseline and learned richer, state-dependent policies that use all four available actions — capturing the nuance of real adjudication decisions that binary supervised classifiers cannot represent.

Overall Conclusions & Cross-Paradigm Comparison

Synthesis

This project applied the full spectrum of machine learning paradigms to a high-dimensional, real-world insurance claims dataset. Each paradigm contributed complementary insights:

Paradigm	Key Contribution	Primary Limitation
Unsupervised	3 natural claim groupings; data structure; guided preprocessing.	Clusters do not align cleanly with target labels.
Supervised	ROC-AUC 0.725 with XGBoost; behavioral features as primary predictors.	Binary label restricts action richness; model is static.
Reinforcement	Adaptive multi-action policy learning; richer decision-making.	Higher computational cost; reward function design is subjective.

Lessons Learned

- High dimensionality with weak individual-feature signal is not insurmountable — careful preprocessing and powerful ensemble methods can extract useful predictive structure.
- The choice of evaluation metric fundamentally shapes what a model learns. PR-AUC is more informative than accuracy in imbalanced settings.
- Preprocessing decisions — especially the choice between RobustScaler and StandardScaler — can dramatically affect downstream results.
- Reinforcement learning introduces a new design dimension (reward function) that encodes business priorities explicitly, making domain expertise a first-class input.
- Model-based RL (Value Iteration) provides an upper bound for model-free methods; the gap between Value Iteration and Q-Learning/SARSA quantifies the cost of not knowing the transition model.

Recommendations for Deployment

- Deploy XGBoost as a real-time triage classifier to flag high-confidence acceleration candidates, with a configurable probability threshold.
- Use the 3-cluster segmentation to create claim profiles for portfolio reporting and adjuster training.
- Invest in reward function calibration for the RL agent using historical cost data to enable production deployment of multi-action adjudication automation.

Together, the three paradigms form a coherent analytical framework: unsupervised learning structures the data, supervised learning makes predictions, and reinforcement learning operationalizes those predictions into business decisions — each building on the previous while offering independent value.